

Compiler Design and Language Processors

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Compiler Design and Language Processors — One-Liners	3
1. Language Processing System	3
2. Assemblers	3
3. Interpreters	3
4. Loaders and Linkers	3
5. Macro Processing	4
6. Compiler Phases, Passes, and Tools	4
7. Lexical Analysis	4
8. Syntax Analysis — Grammars	5
9. Syntax Error Handling	5
10. Top-Down Parsing and FIRST/FOLLOW	5
11. Bottom-Up and LR Parsing	6
12. Semantic Analysis and Syntax-Directed Translation	6
13. Intermediate Code Representation	6
14. Storage Organization	7
15. Code Optimization	7
16. Code Generation	7
17. Exam Pointers Recap	7

Compiler Design and Language Processors — One-Liners

Rapid-revision, double-layered one-liners for the **IBPS SO (IT Officer)** exam. Every fact pairs a count/metric with the exact names so it is exam-ready.

1. Language Processing System

1. There are **3** classical translators: the **compiler**, the **interpreter**, and the **assembler**.
2. A preprocessor performs **4** main tasks: **macro expansion**, **file inclusion**, **conditional compilation**, and **rational preprocessing**.
3. The full language-processing pipeline has **5** stages in order: **preprocessor**, **compiler**, **assembler**, **linker**, and **loader**.
4. A compiler processes the program in **1** unit (the **whole program**), whereas an interpreter processes **1 statement at a time**.
5. Run-time execution is **faster** with a **compiler** because source is translated **only once**, unlike the **interpreter**.
6. The assembler takes **1** input type — **assembly language** (mnemonics) — and produces **machine/object code**.
7. Java is **both** compiled and interpreted: **javac** emits **bytecode** and the **JVM** then interprets/JIT-compiles it.
8. Debugging is **easier** in exactly **1** translator — the **interpreter** — because it stops at the **first error**.
9. The **2** language types that a compiler bridges are a **high-level language** (input) and **machine/target code** (output).

2. Assemblers

10. An assembler performs **5** core functions: **mnemonic-to-opcode** translation, **address assignment**, **symbol-table building**, **directive processing**, and **object-code generation**.
11. There are **2** kinds of assembler by traversal count: the **single-pass** assembler and the **two-pass** assembler.
12. A two-pass assembler uses **2** passes: **Pass 1** builds the **SYMTAB** and assigns addresses, and **Pass 2** generates **object code**.
13. The **forward-reference** problem is naturally solved by the **two-pass** assembler, since all symbols are known after **Pass 1**.
14. A single-pass assembler handles forward references using **1** technique: **backpatching** (fill the address later).
15. An assembler maintains **4** key tables: **OPTAB**, **SYMTAB**, **LITAB**, and the **POT** (pseudo-op table).
16. The **OPTAB** stores **2** things per mnemonic: its **binary opcode** and its **instruction length**, and is used in **both** passes.
17. The **SYMTAB** is built in **Pass 1** and stores **1** mapping: each **label** to its **address**.
18. The **location counter (LC)** serves **1** purpose: assigning **sequential addresses** to instructions and data.
19. A SIC object program has **4** record types: **Header (H)**, **Text (T)**, **Modification (M)**, and **End (E)**.
20. Common assembler directives include **START**, **END**, **RESW**, **RESB**, **BYTE**, **WORD**, and **EQU** — all handled via the **POT**.

3. Interpreters

21. An interpreter offers **3** main advantages: **easier debugging**, **portability**, and a **smaller memory footprint** (no object file).
22. An interpreter has **2** main disadvantages: **slower execution** and **repeated re-translation** of loop bodies.
23. An interpreter produces **0** object-code files because it executes the source **directly**.
24. The **1** situation favoring an interpreter is **rapid development/scripting**, valuing **portability and debugging** over speed.

4. Loaders and Linkers

25. A loader performs **4** functions: **Allocation**, **Linking**, **Relocation**, and **Loading** (mnemonic **A-L-R-L**).
26. There are **5** loader schemes: **absolute**, **bootstrap**, **relocating (BSS)**, **direct-linking**, and **dynamic loading**.
27. The **absolute loader** performs **0** relocation and **0** linking — code loads at a **fixed address**, making it the **simplest**.
28. The **bootstrap loader** runs exactly **1** time — at **power-on** — and its job is to load the **operating system**.

29. The **relocating (BSS) loader** uses **1** mechanism — **relocation bits** — so a program can be loaded **anywhere** in memory.
30. The **direct-linking loader** uses **4** record types: **ESD**, **TXT**, **RLD**, and **END**.
31. **Dynamic loading** loads a routine at exactly **1** moment: **when it is first called**, improving memory use.
32. A linker resolves **1** core problem: **external references** (a symbol defined in one module, used in another).
33. There are **2** linking schemes: **static linking** (copied at link time) and **dynamic linking** (shared at run time).
34. **Static linking** yields a **larger** executable, while **dynamic linking** yields a **smaller** one by **sharing** libraries.
35. Dynamic linking is deferred to **load/run time** and is implemented via **shared libraries** — **DLL** on Windows and **.so** on Linux.
36. **Relocation** can be performed by any of **3** components: the **assembler**, the **linker**, or the **loader**.

5. Macro Processing

37. A macro definition is delimited by **2** directives: **MACRO** (begin) and **MEND** (end).
38. A macro lifecycle has **3** steps: **definition**, **call (invocation)**, and **expansion**.
39. A macro processor uses **3** databases: the **MNT** (Macro Name Table), the **MDT** (Macro Definition Table), and the **ALA** (Argument List Array).
40. Conditional macro expansion uses **3** facilities: **AIF** (conditional branch), **AGO** (unconditional branch), and **SET** symbols.
41. A **nested macro** has **1** defining feature: a **macro call inside another macro definition**.
42. Compared on **3** axes, a macro is expanded **in-line** (no control transfer), making code **larger** but **faster** than a subroutine.
43. A subroutine keeps **1** copy of the code and uses **CALL/RETURN**, trading speed for **smaller** object size.
44. A two-pass macro processor uses **2** passes: **Pass 1** recognizes definitions (build **MNT/MDT**) and **Pass 2** expands calls.

6. Compiler Phases, Passes, and Tools

45. A compiler has **6** phases: **lexical**, **syntax**, **semantic** analysis, **intermediate code generation**, **optimization**, and **code generation**.
46. A compiler is split into **2** halves: the **front end (analysis)** and the **back end (synthesis)**.
47. The **front end** contains **3+** source-dependent phases — **lexical**, **syntax**, **semantic** — plus intermediate code generation.
48. The **back end** contains **2** machine-dependent phases: **machine-dependent optimization** and **code generation**.
49. Lexical analysis takes **1** input (a **character stream**) and produces **1** output (a **token stream**).
50. Syntax analysis takes **tokens** and produces **1** output: a **parse tree / syntax tree**.
51. Semantic analysis performs **1** signature task — **type checking** — and outputs an **annotated tree**.
52. Exactly **2** components span all phases: the **symbol table manager** and the **error handler**.
53. There are **2** compiler organizations by traversal: the **single-pass** compiler and the **multi-pass** compiler.
54. A **pass** differs from a **phase** in **1** way: a pass is a **physical traversal** while a phase is a **logical sub-task**.
55. The **2** classic compiler-construction tools are **LEX** (scanner generator) and **YACC** (parser generator).
56. **LEX** generates a scanner from **regular expressions** and emits the function **yylex**, while **YACC** emits **yparse**.
57. **YACC** builds exactly **1** kind of parser: an **LALR(1)** parser.

7. Lexical Analysis

58. The lexical analyzer has **3** key terms: **token**, **pattern**, and **lexeme**.
59. A **token** is a **pair** with **2** parts: a **token-name** and an optional **attribute value**.
60. A **pattern** is **1** thing — a **rule (regular expression)** — describing all lexemes of a token.
61. A **lexeme** is **1** thing — the **actual character sequence** in the source matching a pattern.
62. The scanner additionally removes **2** items the parser never sees: **white-space** and **comments**.
63. Input buffering uses **2** buffer halves of **N** characters each, called a **buffer pair**.
64. The scanner tracks the current lexeme with **2** pointers: **lexemeBegin** and **forward**.

65. A **sentinel** is **1** special character — **eof** — placed at the **end of each buffer half** to cut comparisons.
66. Regular-expression operators have **3** precedence levels: **Kleene star (*)** highest, then **concatenation**, then **union (|)** lowest.
67. The token-recognition tool chain has **3** conversions: **RE to NFA**, **NFA to DFA**, and **DFA minimization**.
68. **RE-to-NFA** uses **1** algorithm — **Thompson's construction** — which introduces **ϵ -transitions**.
69. **NFA-to-DFA** uses **1** algorithm — **subset construction** — with a worst case of **2^n** DFA states.
70. **DFA minimization** by **Hopcroft's algorithm** runs in **$O(n \log n)$** time.
71. NFA and DFA recognize exactly **1** identical class of languages: the **regular** languages.
72. The **2** differences of DFA from NFA: **no ϵ -moves** and exactly **1 transition** per input symbol.

8. Syntax Analysis — Grammars

73. A **Context-Free Grammar** is a **4-tuple (V, T, P, S)**: non-terminals, terminals, productions, and start symbol.
74. There are **2** standard derivations: **leftmost (LMD)** and **rightmost (RMD / canonical)**.
75. A parse tree has **2** node kinds: **terminals as leaves** and **non-terminals as internal nodes**.
76. A grammar is **ambiguous** when **1** string has **more than 1** parse tree (or leftmost derivation).
77. **Left recursion** $A \rightarrow A\alpha \mid \beta$ is removed by **2** rules: $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$.
78. **Left factoring** removes **1** thing — a **common prefix** — turning $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ into $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$.
79. The **Chomsky hierarchy** has **4** types: **Type 0**, **Type 1**, **Type 2**, and **Type 3**.
80. **Type 0** (unrestricted) is recognized by **1** machine: the **Turing Machine**.
81. **Type 1** (context-sensitive) is recognized by the **Linear Bounded Automaton (LBA)**.
82. **Type 2** (context-free) is recognized by the **Pushdown Automaton (PDA)** and describes programming-language **syntax**.
83. **Type 3** (regular) is recognized by the **Finite Automaton** and describes **tokens**.
84. The hierarchy nests in **1** chain: **Type 3 $\subset$ Type 2 $\subset$ Type 1 $\subset$ Type 0**.

9. Syntax Error Handling

85. There are **4** error-recovery strategies: **panic-mode**, **phrase-level**, **error productions**, and **global correction**.
86. **Panic-mode** recovery skips input until **1** thing appears — a **synchronizing token** like **;** or **}**.
87. **Phrase-level** recovery performs **1** action: a **local correction** on the remaining input.
88. **Global correction** seeks **1** optimum — the **minimum number of changes** — but is the **costliest** strategy.

10. Top-Down Parsing and FIRST/FOLLOW

89. There are **2** styles of top-down parsing: **recursive-descent** and **predictive (LL(1))** parsing.
90. Top-down parsing builds the tree from **root to leaves**, tracing a **leftmost derivation**.
91. Naive recursive descent may need **1** costly mechanism — **backtracking** — and cannot handle **left recursion**.
92. **LL(1)** expands to **3** facts: scan **Left-to-right**, build **Leftmost** derivation, with **1** lookahead symbol.
93. A predictive parser needs **2** grammar fixes: **no left recursion** and **left factoring**.
94. A predictive parsing table is indexed by **2** keys: a **non-terminal (row)** and a **terminal (column)**, as $M[A, a]$.
95. A grammar is **LL(1)** iff for $A \rightarrow \alpha \mid \beta$, $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ and (if $\epsilon \in \text{FIRST}(\beta)$) $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$.
96. **FIRST(X)** for a terminal X has exactly **1** member: **{X}** itself.
97. **FOLLOW(S)** of the start symbol always contains **1** mandatory symbol: the end-marker **\$**.
98. Exactly **1** symbol is never placed in any FOLLOW set: **ϵ** ; and **\$** is never placed in any FIRST set.
99. A **left-recursive or ambiguous** grammar can be LL(1) in **0** cases (never).
100. For the classic grammar, $\text{FIRST}(E) = \{ (, id \}$ and $\text{FOLLOW}(E) = \{), \$ \}$.
101. For the classic grammar, $\text{FIRST}(E') = \{ +, \epsilon \}$ and $\text{FOLLOW}(T) = \{ +,), \$ \}$.

11. Bottom-Up and LR Parsing

102. Bottom-up parsing builds the tree from **leaves to root**, tracing a **rightmost derivation in reverse**.
103. A **handle** is **1** substring matching a production body whose **reduction** is one reverse-rightmost step.
104. **Handle pruning** repeats **1** action: locating a handle and **reducing** it.
105. A shift-reduce parser has **4** actions: **shift**, **reduce**, **accept**, and **error**.
106. A shift-reduce parser uses **1** data structure — a **stack** — plus an input buffer.
107. **Operator-precedence** parsing uses **3** relations: **<·** (yields), **=·** (equal), and **·>** (takes precedence).
108. An **operator grammar** has **2** restrictions: **no two adjacent non-terminals** and **no ϵ -production**.
109. **LR(k)** expands to **3** facts: scan **Left-to-right**, build **Rightmost** derivation in reverse, with **k** lookahead.
110. The **LR** family has **4** members: **LR(0)**, **SLR(1)**, **LALR(1)**, and **CLR(1) / LR(1)**.
111. An LR parser is driven by **2** tables: the **ACTION** table and the **GOTO** table.
112. LR parsing starts from **1** transformation — the **augmented grammar** $S' \rightarrow S$.
113. **SLR(1)** decides reductions using **1** set — the **FOLLOW** set — over **LR(0)** items.
114. **CLR(1) / LR(1)** is the **most powerful** LR parser and has the **most** states.
115. **LALR(1)** is built by **1** operation: **merging LR(1) states with the same core (LR(0) items)**.
116. The parsing-power order is **1** chain: **LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ CLR(1)**.
117. By state count, **SLR(1) = LALR(1) < CLR(1)** — SLR and LALR have the **same** number of states.
118. There are **2** LR conflict types: **shift-reduce (S-R)** and **reduce-reduce (R-R)**.
119. The **dangling-else** is the classic **shift-reduce** conflict, resolved by **preferring shift**.
120. State merging in LALR can introduce **1** new conflict type — **reduce-reduce** — but **never** a new **shift-reduce** conflict.
121. Every **LL(1)** grammar is also **LR(1)**, but the reverse holds in **0** general cases.

12. Semantic Analysis and Syntax-Directed Translation

122. Semantic analysis performs **4** checks: **type checking**, **scope resolution**, **declaration-before-use**, and **argument/bound consistency**.
123. There are **2** syntax-directed mechanisms: the **SDD** (definition) and the **SDT** (translation with actions).
124. There are **2** attribute kinds: **synthesized** (from children) and **inherited** (from parent and siblings).
125. **Synthesized** attributes are evaluated **bottom-up** and fit **1** parser family — **LR**.
126. **Inherited** attributes fit **1** parser family — **LL** — and L-attributed definitions allow them.
127. An **S-attributed** definition uses **only synthesized** attributes and needs exactly **1** bottom-up pass.
128. An **L-attributed** definition evaluates in **1** left-to-right pass, depending only on **parent + left siblings**.
129. A terminal/leaf can hold a **synthesized** attribute but **0** inherited attributes.
130. Among the **2** classes, **every S-attributed grammar is also L-attributed**, but not vice-versa.

13. Intermediate Code Representation

131. There are **4** common intermediate-code forms: **syntax tree/AST**, **DAG**, **three-address code**, and **postfix notation**.
132. An **AST** has **2** node roles: **operators as internal nodes** and **operands as leaves**.
133. A **DAG** differs from an AST by **1** feature: it **shares common subexpressions** (one node per repeated computation).
134. **Three-address code** has at most **1** operator and at most **3** addresses per instruction, form **$x = y \text{ op } z$** .
135. Three-address code has **3** implementations: **quadruples**, **triples**, and **indirect triples**.
136. A **quadruple** has **4** fields: **op**, **arg1**, **arg2**, and **result**.
137. A **triple** has **3** fields: **op**, **arg1**, **arg2** — the result is referenced by its **position**.
138. **Indirect triples** add **1** structure — a **list of pointers** to triples — easing reordering.
139. For optimization, **quadruples** are the **easiest** to reorder while **triples** are the **hardest** (position-dependent).
140. **Postfix (Reverse Polish)** notation needs **0** parentheses; e.g., **$a + b * c$** becomes **$a \ b \ c \ * \ +$** .

14. Storage Organization

- 141. Run-time memory has **4** regions: **code**, **static data**, **heap**, and **stack**.
- 142. Of the dynamic regions, the **stack grows downward** and the **heap grows upward** (toward each other).
- 143. The **stack** stores **1** kind of structure — **activation records** — managed in **LIFO** order.
- 144. The **heap** holds data with **1** property: it may **outlive the call** that created it, needing **garbage collection**.
- 145. An activation record has **7** typical fields including **return value**, **actual parameters**, **control link**, **access link**, **saved status**, **locals**, and **temporaries**.
- 146. The **control (dynamic) link** points to the **caller's** record, while the **access (static) link** is for **non-local** data.
- 147. There are **3** storage-allocation strategies: **static**, **stack**, and **heap**.
- 148. **Static allocation** supports recursion in **0** cases (no), since memory is fixed at **compile time**.
- 149. **Stack allocation** supports recursion in **all** cases, allocating a record **per activation**.
- 150. There are **4** parameter-passing methods: **call-by-value**, **call-by-reference**, **call-by-value-result**, and **call-by-name**.
- 151. **Call-by-value** passes **1** thing — a **copy** — so the caller sees **0** changes.
- 152. **Call-by-reference** passes the **address (l-value)** and reflects changes **immediately**.
- 153. **Call-by-value-result** copies back at exactly **1** moment: on **return** (copy-restore).
- 154. **Call-by-name** uses **1** technique — **textual substitution** (lazy re-evaluation) — as in **Algol-60**.

15. Code Optimization

- 155. Optimization is of **2** kinds: **machine-independent** (on IR) and **machine-dependent** (on target code).
- 156. A **basic block** has exactly **1** entry (the **leader**) and **1** exit, with no internal branching.
- 157. **Leaders** are identified by **3** rules: the **first statement**, any **jump target**, and any statement **after a jump**.
- 158. A **flow graph** has **2** components: **basic blocks as nodes** and **control flow as edges**.
- 159. Optimization scope is **2-fold**: **local** (within one block) and **global** (across blocks via **data-flow analysis**).
- 160. **Constant folding** evaluates **1** thing — **constant expressions** — at compile time, e.g., $3 * 4$ becomes **12**.
- 161. **Common subexpression elimination** ensures a repeated expression like $a + b$ is computed exactly **1** time.
- 162. **Dead-code elimination** removes code used in **0** places (never referenced).
- 163. **Strength reduction** replaces **1** costly operation with a cheaper one, e.g., $x * 2$ becomes $x + x$ or a **shift**.
- 164. **Loop-invariant code motion** moves unchanging code **out** of the loop, executing it **1** time instead of every iteration.
- 165. **Peephole optimization** scans **1** small **sliding window** of target instructions to replace them with faster ones.
- 166. Peephole optimization includes **5** techniques: **redundant load/store removal**, **flow-of-control optimization**, **algebraic simplification**, **strength reduction**, and **machine idioms**.

16. Code Generation

- 167. Code generation is the **last** of the **6** phases and produces **target/machine code** from optimized IR.
- 168. Code generation faces **4** key issues: **input form**, **target-program form**, **instruction selection**, and **register allocation/assignment**.
- 169. The target program may take **3** forms: **absolute**, **relocatable**, or **assembly** code.
- 170. There are **2** distinct register tasks: **allocation** (which values go in registers) and **assignment** (which specific register).
- 171. Optimal **register allocation** has **1** complexity class — it is **NP-complete**.
- 172. A simple code generator uses **2** aids: a **getReg()** function and **next-use information**.
- 173. Code generation belongs to the **back end**, making it **machine-dependent** and thus **not** portable.

17. Exam Pointers Recap

- 174. The fixed phase order has **6** steps: **lexical** → **syntax** → **semantic** → **intermediate** → **optimization** → **code generation**.
- 175. Exactly **1** phase removes comments/white-space — **lexical analysis** — and **1** phase does type checking — **semantic analysis**.

176. The **2** parser paradigms map cleanly: **LL = top-down/predictive** and **LR (SLR/LALR/CLR) = bottom-up**.
177. The **2** confusable tools: **LEX = RE-based scanner** and **YACC = LALR(1) parser generator**.
178. For forward references, **two-pass assembler** solves them directly while **single-pass** needs **backpatching**.
179. The **2** distinct system tasks: a **linker** resolves **external references**; a **loader** brings code **into memory**.
180. The **3** three-address-code forms rank by reordering ease: **quadruples (easy) > indirect triples > triples (hard)**.
181. Both **NFA and DFA** accept the **same** regular languages, but subset construction may need **2^n** states.
182. The power chain for LR is **LR(0) < SLR(1) < LALR(1) < CLR(1)**, while states obey **SLR = LALR < CLR**.
183. A CFG is a **4-tuple (V, T, P, S)** and sits at **Type 2** of the **4-level** Chomsky hierarchy.
184. There are **2** parse-tree node kinds and **2** derivation orders (**leftmost, rightmost**) for any CFG.
185. The end-marker **\$** appears only in **FOLLOW**, never in **FIRST**, and **ε** never appears in any **FOLLOW** set.
186. There are **4** loader functions (**A-L-R-L**) and **5** loader types, of which only the **bootstrap loader** loads the **OS**.
187. There are **3** macro-processor databases (**MNT, MDT, ALA**) and **3** conditional facilities (**AIF, AGO, SET**).
188. A macro trades **space for time** by in-line expansion, while a subroutine trades **time for space** via **CALL/RETURN**.
189. The **2** semantic attribute classes are **synthesized** (LR/bottom-up) and **inherited** (LL/top-down).
190. There are **3** allocation strategies, and only **2** of them — **stack** and **heap** — support **recursion/dynamic** data.